

Reinforcement Learning for Large-Scale Auto-Bidding

Authors. (α - β order) Ruirong Feng (rrfeng@princeton.edu), Ophelia Fulton (of4642@princeton.edu), Xinya (Claire) Gu (xg3412@princeton.edu), Bonnie Liu (bl0919@princeton.edu), Yiqi (Andrew) Liu (andrew.liu@princeton.edu)

Link to GitHub Repo. <https://github.com/liuyiqiandrew/AuctionNet>

1 Introduction

Auto-bidding is the control layer between an advertiser’s campaign objective and the real-time auction market. An advertiser aims to maximize conversions, under a limited budget and the realized cost per acquisition (CPA). AuctionNet turns this problem into a public benchmark built from large-scale auction logs: at each decision point, the bidder chooses a scalar multiplier that determines bids for a batch of impressions, competes against other advertisers, and receives reward only after auction outcomes and conversions are realized [7, 9]. The task is therefore a natural sequential decision problem, but not a clean textbook one. The agent observes only a summary of the market, the relevant constraints operate over an episode, and the best action depends on how much budget and CPA slack should be saved for future traffic.

Static or myopic bidding rules miss this tradeoff. A policy that bids aggressively on every high-probability impression can exhaust its budget before later opportunities arrive; a policy that only protects CPA can leave valuable conversions unclaimed. Standard on-policy actor-critic training is a natural baseline because it learns directly from simulator feedback, but the naive version leaves several failure modes exposed: the CPA penalty is delayed, the observation hides market-regime changes, and uniform rollouts spend the same gradient budget on easy and hard periods.

Recent alternatives make larger changes to the solution concept. Dynamic-knapsack methods decompose long-horizon advertising into budget allocation and policy optimization, but depend on a knapsack structure and greedy selection rule that is not the same as learning a scalar bid multiplier inside AuctionNet’s replay simulator [5]. DiffBid avoids the Markov assumption by training a conditional diffusion model over whole trajectories, but this shifts the problem toward offline trajectory generation from historical bidding logs rather than online policy improvement [4]. Oracle imitation learning is especially strong on AuctionNet, but its supervision comes from a hindsight oracle that sees the full campaign traffic and solves a nonlinear multiple-choice knapsack problem before training the student policy [2].

These methods identify what vanilla online RL does not specify on its own: how delayed CPA feedback should shape pacing, which market statistics should be exposed to a partially observed policy, how recent history should enter the state, and how training updates should be allocated across heterogeneous periods. We study these factors while keeping the AuctionNet scalar action, replay simulator, and evaluation protocol fixed. Starting from the provided actor-critic policy, we formulate the task as a POMDP and use the same baseline pipeline as the reference point for all extensions [1, 6, 8].

Our experiments vary the components of the online RL pipeline rather than replacing the benchmark with a different supervision problem. At the observation level, we compare compact benchmark features, a richer state used by stronger competition agents, and additional market-regime signals. At the training level, reward shaping and score-weighted period sampling test whether the policy benefits from denser constraint feedback and more informative rollouts. At the policy level, a recurrent encoder tests whether learned history summaries improve over hand-crafted rolling statistics. We also evaluate large language models as bidding policies by fine-tuning Qwen3-8B with GRPO and LoRA adapters. Across these experiments, improvements in state design account for the largest

gains, while the pretrained LLM policy shows faster early adaptation but a lower ceiling than the strongest non-LLM actor-critic policy in our runs. The resulting study provides a compact training recipe for constrained decision making under partial information and an empirical probe of how generic pretrained policies behave in numerical constrained optimization with delayed rewards.

2 Methods

2.1 Problem Setup and PPO Baseline

We study the AuctionNet auto-bidding environment from the NeurIPS 2024 competition on auto-bidding under uncertainty [9]. The environment follows the real-time auction setting used in recent auto-bidding work [2]. Each episode corresponds to one advertiser in one delivery period. The advertiser is given a total budget B , a target cost per acquisition $\text{CPA}_{\text{target}}$, and a horizon of $T = 48$ bidding ticks. At tick t , the simulator presents a batch $\mathcal{I}_t$ of impression opportunities, each with a predicted conversion probability $p_{t,i}$. The agent chooses a scalar log-bid multiplier $a_t \in [-10, 10]$, which is applied to every impression in the batch:

$$b_{t,i} = \exp(a_t) p_{t,i} \text{CPA}_{\text{target}}, \quad i \in \mathcal{I}_t.$$

The controlled bids are inserted into historical auctions against the other 47 advertisers, whose bids are fixed by the dataset. Slots and prices are then determined by the auction simulator, spend is deducted from the remaining budget, and conversions are sampled from the predicted values. The simulator enforces the budget constraint, while the AuctionNet evaluation score applies a quadratic penalty when the realized CPA exceeds the target. For a trajectory τ , total conversions and cost are $Y_T(\tau)$ and $C_T(\tau)$, respectively, and the realized CPA is $\text{CPA}_T(\tau) = C_T(\tau) / \max\{Y_T(\tau), 1\}$. The evaluation score is

$$R(\tau) = Y_T(\tau) \cdot \min \left\{ 1, \left(\frac{\text{CPA}_{\text{target}}}{\text{CPA}_T(\tau)} \right)^2 \right\}.$$

We formulate the task as a partially observable Markov decision process $(\mathcal{S}, \mathcal{A}, \Omega, \mathcal{P}, \mathcal{O}, \mathcal{R})$. The latent state $s_t \in \mathcal{S}$ contains the current campaign status, auction traffic, competitor bids, and future market conditions that the agent cannot fully observe. The action space $\mathcal{A}$ is the scalar log-bid multiplier above. The observation $o_t \in \Omega$ is a feature vector summarizing the current tick, remaining budget, target CPA, current impression statistics, and historical bidding outcomes. After the agent samples $a_t \sim \pi_\theta(\cdot | h_t)$, with history $h_t = (o_0, a_0, \dots, o_t)$, the simulator transitions according to $\mathcal{P}(\cdot | s_t, a_t)$, emits a new observation through $\mathcal{O}$, and returns reward $r_t = \mathcal{R}(s_t, a_t)$. In the standard PPO baseline, this reward is the per-step CPA-clipped score computed from the realized cost and conversions at tick t ; the terminal AuctionNet score is used for evaluation. The expected return under policy π_θ is

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right],$$

with discount factor γ . Our baseline uses the 16-feature observation, an MLP actor-critic policy with a diagonal Gaussian action distribution, and running observation/reward normalization. It is trained on 20 periods of trajectory data and evaluated on one held-out period.

For the baseline actor-critic policy, we optimize the standard clipped PPO surrogate. Let $\pi_{\theta_{\text{old}}}$ be the rollout policy, $\rho_t(\theta) = \pi_\theta(a_t | o_t) / \pi_{\theta_{\text{old}}}(a_t | o_t)$, $\hat{A}_t$ be the advantage estimate, and $\hat{V}_t$ be the bootstrapped return target. The PPO objective is

$$\begin{aligned} \mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_t \Big[& \min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon_c, 1 + \epsilon_c) \hat{A}_t) \\ & - c_v (V_\theta(o_t) - \hat{V}_t)^2 + c_H \mathcal{H}(\pi_\theta(\cdot | o_t)) \Big]. \end{aligned} \quad (1)$$

Beyond the standard clipped PPO surrogate, the quadratic term fits the value function used for advantage estimation, and the entropy term $\mathcal{H}(\pi_\theta)$ discourages premature collapse

of the continuous Gaussian policy. This baseline fixes the auction simulator, scalar bid interface, and evaluation protocol for all PPO-based extensions below.

2.2 PPO Pipeline Extensions

We treat the online PPO bidder as the reference pipeline and vary the components most directly tied to constrained bidding under partial information: constraint feedback, rollout allocation, temporal context, and market state. All variants share the AuctionNet scalar action interface and replay evaluation protocol, yielding a controlled ablation in which score differences are attributable to the modified component.

Reward Shaping. The unconstrained PPO maximizes rewards but without mechanism to constrain CPA when the agent exceeds the target CPA. We added a fixed Lagrangian penalty coefficient λ that penalizes CPA exceedance, applied per step rather than only at terminal:

$$\mathcal{L}(\pi, \lambda) = \mathbb{E} \left[\sum_t \text{conv}_t \right] - \lambda \cdot \mathbb{E} \left[(\text{total_cost} - \text{target_CPA} \cdot \text{total_conv})_+ \right]$$

λ here is a hyperparameter that we tune, and more specifically $\lambda \in \{.0001, .001, .01\}$. We calculated the per step overspend as:

$$\text{overspend}_t = \max(0, \text{cost}_t - \text{target_cpa} \cdot \text{conv}_t)$$

so the shaped reward $r_t^{\text{shaped}} = r_t - \lambda * \text{overspend}_t$, and when $\lambda = 0$, we have the original PPO. With such, we expect the policy learns to bid more carefully and follows CPA in training.

Sample Reweighting. The default pipeline samples uniformly from training periods 7–26, spending equal gradient budget on periods where the policy already performs well and those it struggles with. We replace this with a score-weighted distribution that oversamples high-performing (advertiser, period) pairs. For each pair j we compute the policy’s NeurIPS score and assign sampling weight

$$w_j = \alpha \cdot \text{softmax}(\text{score}_j / T) + (1 - \alpha) \cdot \frac{1}{N}, \quad (2)$$

where T controls concentration strength and α interpolates between reweighted and uniform sampling. At each rollout, the worker draws a new pair according to w_j . We set $\alpha=0.9$ and $T=30$: the $(1-\alpha)$ uniform floor ensures every period is visited occasionally, preventing the policy from forgetting rare regimes, while $T=30$ provides moderate concentration, which ensures the highest-scoring trajectories receive several times the sampling probability of the lowest, but without collapsing onto a small handful of easy campaigns. All other pipeline components remain unchanged.

Temporal Encoding. The original online PPO bidding policy observes each auction period as a state vector $x_t \in \mathbb{R}^d$ and uses a feed-forward network to map the state vector to a bid coefficient. The temporal extension alters the state representation for the policy by using the K most recent states as the input X_t for each step. If fewer than K steps are available, the history is padded by repeating the earliest available state.

The history X_t is encoded into a hidden summary h_t by a residue Gated Recurrent Unit (GRU) [3]. The hidden vector h_t is concatenated with the latest state x_t and passed through layer normalization to form the augmented PPO input z_t .

$$h_t = \text{GRU}(X_t), \quad z_t = \text{LayerNorm}([h_t; x_t]). \quad (3)$$

This residue path preserves current-period bidding features, while the GRU summarizes short-horizon trajectory information.

Unless otherwise specified, actor and critic use separate temporal feature extractors, matching the intuition that action selection and value estimation may attend to different temporal summaries. A shared-extractor variant was also implemented to test whether the extra capacity of separate GRUs is necessary or whether a single temporal representation regularizes the actor-critic update.

Market-aware Feature Engineering.**2.3 RL Training on LLM**

We test the performance of large language models (LLM) on the auto-bidding task before and after RL training to explore whether a pretrained generic model can improve final score, sample efficiency, and training stability of reinforcement learning. We use Qwen3-8B as the base policy model. At each tick, the current auction state is serialized into a short prompt containing budget, target CPA, market statistics, and historical performance. The model emits one numeric action, treated as the same log-bid multiplier a_t used by the PPO baseline:

$$b_{t,i} = \exp(a_t) p_{t,i} \text{CPA}_{\text{target}}.$$

We adopt Group Relative Policy Optimization (GRPO) as the training objective. For each prompt x , the rollout policy π_{old} samples G trajectories $\{\tau_i\}_{i=1}^G$, receives terminal returns $\{R_i\}_{i=1}^G$, and assigns each trajectory the group-normalized advantage $\hat{A}_i = (R_i - \mu_R) / (\sigma_R + \epsilon)$, where μ_R and σ_R are computed within the group. The optimization objective is the clipped surrogate with a KL penalty to the reference policy:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_{x, \{\tau_i\}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(\frac{\pi_{\theta}(\tau_i | x)}{\pi_{\text{old}}(\tau_i | x)} \hat{A}_i, \right. \right. \\ \left. \left. \text{clip} \left(\frac{\pi_{\theta}(\tau_i | x)}{\pi_{\text{old}}(\tau_i | x)}, 1 - \epsilon_c, 1 + \epsilon_c \right) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}}) \right]. \quad (4)$$

In the auto-bidding environment, a prompt x corresponds to one (period, advertiser) row. One trajectory τ_i is a full 48-tick episode: at every tick, the environment state is rendered as the next user message, the LLM emits the numeric action a_t , and the simulator advances one step. Chain-of-thought is disabled so that the assistant tokens are only the action response. Each GRPO update samples four prompts and $G = 4$ trajectories per prompt, giving 16 episodes whose terminal AuctionNet scores are the returns R_i above. During training, we use LoRA adapters on Qwen3-8B’s attention and MLP projection modules to make policy updates feasible without full-parameter fine-tuning. The experiment keeps the AuctionNet dynamics and scalar bid interface fixed relative to the MLP actor-critic baseline, isolating the effect of replacing that policy class with an LLM.

3 Results**3.1 Temporal Encoding**

Table 1: Held-out period-27 advertiser-sweep score for temporal encoding extension. Entries report mean terminal scores.

Features	K	$H = 32$		$H = 64$	
		Separate	Shared	Separate	Shared
16	Baseline	20.14 ± 2.25			
16	4	23.80 ± 2.32	23.58 ± 2.29	24.78 ± 2.29	24.48 ± 2.27
16	8	24.26 ± 2.46	22.85 ± 2.25	22.69 ± 2.24	23.93 ± 2.17
16	32	22.35 ± 2.38	24.23 ± 2.24	23.97 ± 2.45	22.77 ± 2.37
60	Baseline	23.23 ± 2.02			
60	4	22.07 ± 2.18	23.73 ± 2.40	24.56 ± 2.15	20.45 ± 2.16
60	8	22.18 ± 1.85	24.85 ± 2.28	23.01 ± 2.33	23.38 ± 1.50
60	32	22.94 ± 2.25	18.68 ± 1.45	23.47 ± 2.60	23.10 ± 2.03

The temporal encoder extension is examined over three hyper parameter: history length ($K = 4, 8, 32$), GRU hidden dimension size ($H = 32, 64$), and actor-critic feature sharing

(separated versus shared temporal extractors). Each hyperparameter combination is tested on both the fiducial 16-feature state and the extended 60-feature states for a total of 24 extension experiments.

Table 1 reports the held-out evaluation score for the temporal encoding extension hyperparameter sweep. The extension provides clearest benefit in the fiducial 16-feature observation configurations. The best model ($K = 4$, $H = 64$, separate) improves the evaluation score from 20.14 ± 2.25 to 24.78 ± 2.29 . In contrast, the 60-feature configurations shows limited improvements from temporal encoding. Across hyperparameter variations, the temporal encoding extension yield less systematic effects than the choice of observation feature set.

In the fiducial 16-feature setting, the policy likely benefited from recent historical information encoded by GRU. The extended 60-feature setting, each state already contains historical summary features, making the additional GRU summary partly redundant. In addition, the larger augmented representation may be harder for the actor-critic network of PPO to use effectively, which could explain the underperforming 60-feature variants underperforms the baseline.

3.2 Sample Reweighting

The sample-reweighted PPO achieves a period-27 sweep score of **24.28**, compared to 20.14 for the uniform-sampling baseline which is a gain of 1.0 point (+4.0%). The improvement is consistent across training: the weighted training curve lies above the uniform baseline throughout the full 10M environment steps, not only at convergence (Figure 1). This suggests that the gradient is being allocated more efficiently across the period distribution, rather than the policy simply converging to a different local optimum.

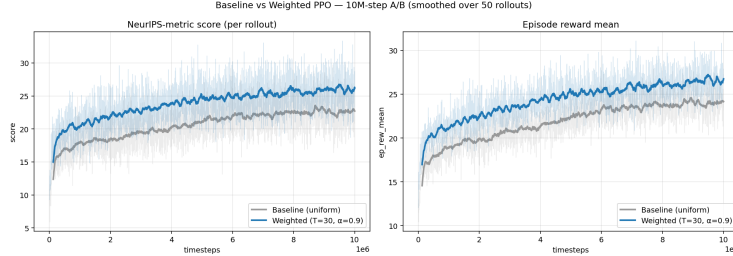


Figure 1: PPO training curves: uniform sampling (baseline) vs. score-weighted period sampling ($\alpha = 0.9$, $T = 30$). Weighted sampling outperforms throughout training.

The result validates the hypothesis that *where* the policy trains matters, not only *how* it trains. By concentrating rollouts on high-scoring pairs, the reweighted sampler lets the policy learn more from demonstrations of effective bidding behavior, i.e. correct pacing, CPA-compliant budget allocation, and selective bid placement, rather than spending equal gradient budget on trajectories that exhibit poor strategies. [OPTIONAL, CAN BE REMOVED: The gain is modest compared to some other extensions (e.g., market-regime conditioning at 29.7), which is expected: sampling changes the *efficiency* of learning but not the *information* available to the policy. The two interventions are complementary and could in principle be stacked.]

[CAN BE PUT INTO 4. DISCUSSION SECTION] One possible explanation for the modest 4% gain is the aggressive choice of $\alpha=0.9$, which places 90% of sampling weight on the softmax distribution. This may cause the policy to overfit to high-scoring campaigns while under-training on harder advertisers with tighter CPA constraints or unusual budget profiles. A lower α (e.g., 0.5–0.7) would retain more uniform coverage and may yield a better balance between learning from good behavior and maintaining robustness across diverse advertiser types. We leave this sweep to future work.

3.3 Market-aware Feature Engineering

@Bonnie: result description+ takeaway (which hypothesis does this extension validates?)

3.4 Reward Shaping

The reward shaping PPO achieves a best sweep score of 25.6, at $\lambda = .0001$ and an improvement from the baseline. The gap for $\lambda = .01$ shows a visible gap in mean episode reward, which reflects the penalty term in the shaped objective and is not a drop in the unshaped score. We bounded sweep λ at .01 as the penalty punishes overspend, and we do not have an additional reward mechanism for efficient bids that are within CPA and converged. A larger λ would bias the policy toward conservative spending, which could hurt conversions that agents would otherwise win. For each unique λ , the agent solved a

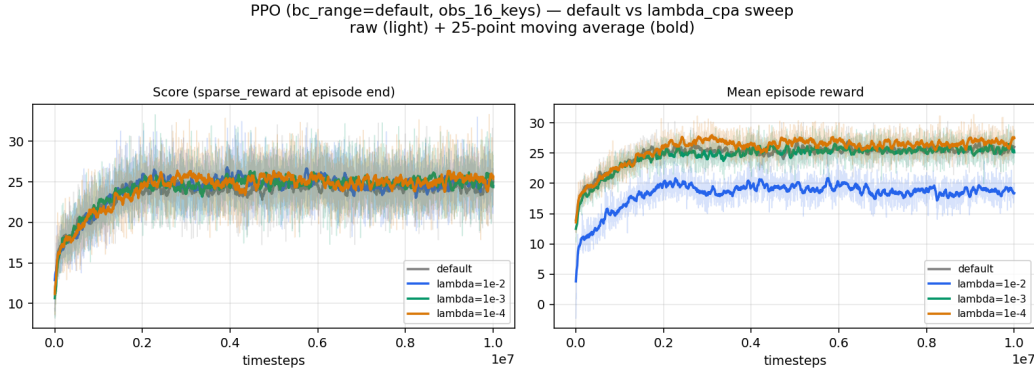


Figure 2: Reward shaping curves across λ values versus baseline PPO. Training results are within similar convergence with the baseline.

different problem than original. We see $\lambda \in \{.0001, .001\}$ is too small to register in training, while $\lambda = .01$ drops training reward by 25% without altering final reward in training. Exploratory runs of stricter λ values, such as .1 or 1, produced a policy toward ineffective bidding.

3.5 RL training on LLM

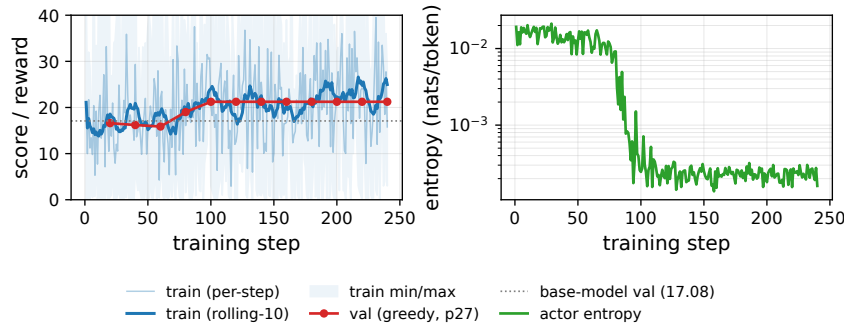


Figure 3: **GRPO + LoRA fine-tuning of Qwen3-8B on AuctionNet periods 7–26 (validation on period 27).** Left: per-step training score (rolling-10 mean and min/max band over each batch of 16 rollouts), greedy validation reward at every checkpoint, and the frozen-base-model validation baseline (gray dotted line at 17.08). Right: actor token entropy on a log scale, tracking how quickly the action distribution concentrates during training.

We report the GRPO training of Qwen3-8B in Figure 3. The run uses rollout temperature 1.0, group size $G = 4$, and no entropy bonus. During training, chain-of-thought generation

was disabled due to the computational resource limit. The validation curve shows the sample efficiency advantage of starting from a pretrained policy: the frozen Qwen3-8B model already reaches a nontrivial validation score of 17.08, and GRPO lifts it to 21 in the early stage of training. The resulting score is below the peak of the best non-LLM actor-critic policies, but the early jump suggests that pretraining gives the policy a useful prior for converting auction states into bid multipliers.

The later training dynamics point to the limitation of this setup. With chain-of-thought disabled, token entropy mainly reflects the spread of the emitted action; once the policy concentrates around a narrow set of bid multipliers, the $G = 4$ samples provide little within-group return variation for GRPO. The current setup therefore reaches a useful low-entropy pacing mode quickly, but larger rollout groups or stronger exploration pressure may be needed to turn the sample-efficiency gain into competitive peak performance.

4 Discussion and Limitations

Several extensions showed better performance than original PPO. The principal limitation is computational resources, since each PPO run trains 10 million steps on the environment. Given the time constraint of the project, hyperparameter tuning is expensive. Future directions include finer hyperparameter tuning and mixed strategy of combining extensions to see if performance gain can be aggregated.

Reward Shaping

References

- [1] Cai, H., Ren, K., Zhang, W., Malialis, K., Wang, J., Yu, Y., and Guo, D. (2017). Real-time bidding by reinforcement learning in display advertising. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining, WSDM 2017*, page 661–670. ACM.
- [2] Chiappa, A. S., Gangopadhyay, B., Wang, Z., and Takamatsu, S. (2025). Auto-bidding in real-time auctions via oracle imitation learning (oil).
- [3] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Moschitti, A., Pang, B., and Daelemans, W., editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar. Association for Computational Linguistics.
- [4] Guo, J., Huo, Y., Zhang, Z., Wang, T., Yu, C., Xu, J., Zheng, B., and Zhang, Y. (2024). Generative auto-bidding via conditional diffusion modeling. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5038–5049.
- [5] Hao, X., Peng, Z., Ma, Y., Wang, G., Jin, J., Hao, J., Chen, S., Bai, R., Xie, M., Xu, M., Zheng, Z., Yu, C., Li, H., Xu, J., and Gai, K. (2020). Dynamic knapsack optimization towards efficient multi-channel sequential advertising.
- [6] He, Y., Chen, X., Wu, D., Pan, J., Tan, Q., Yu, C., Xu, J., and Zhu, X. (2021). A unified solution to constrained bidding in online display advertising. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, KDD '21*, page 2993–3001, New York, NY, USA. Association for Computing Machinery.
- [7] Su, K., Huo, Y., Zhang, Z., Dou, S., Yu, C., Xu, J., Lu, Z., and Zheng, B. (2024). Auctionnet: A novel benchmark for decision-making in large-scale games. *Advances in Neural Information Processing Systems*, 37:94428–94452.
- [8] Wu, D., Chen, X., Yang, X., Wang, H., Tan, Q., Zhang, X., Xu, J., and Gai, K. (2018). Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management, CIKM '18*, page 1443–1451, New York, NY, USA. Association for Computing Machinery.
- [9] Xu, J., Zhang, Z., Lu, Z., Deng, X., Wellman, M. P., Yu, C., Dou, S., Huo, Y., Xu, Z., Duan, Z., Xiong, S., Liu, C., Li, N., Su, K., Gong, W., and Zheng, B. (2024). Auto-bidding in large-scale auctions: Learning decision-making in uncertain and competitive games. In *NeurIPS 2024 Competition Track*.

5 Appendix (Optional)

Include any details that don't fit within the main body here.